

INFO145 - Diseño y Análisis de Algoritmos

10 - PD LCSP e Inducción - Maximum Subarray Sum Problem

Héctor Ferrada
académico

Instituto de Informática
Universidad Austral de Chile

Subsecuencia Comun más Larga (LCS)

Sea una secuencia $X = \langle X_1, X_2, \dots, X_m \rangle$ proveniente de un alfabeto Σ .

- Se define una subsecuencia de X a cualquier secuencia que se forma a partir de X , quitando k símbolos de X ; $0 \leq k \leq m$.
- Se define una subsecuencia común entre dos secuencias X e Y ($CS(X, Y)$), a una subsecuencia de X que también es subsecuencia de Y .
- Se define a la subsecuencia común más larga (LCS), entre X e Y , a la subsecuencia comun entre X e Y que sea de mayor largo.
- Ejem: sean $X = \langle A, B, C, B, D, A, B \rangle$, e $Y = \langle B, D, C, A, B, A \rangle$.
Tenemos que: $\langle B, D \rangle$, $\langle B, C, A \rangle$ y $\langle D, B \rangle$ son ejemplos de subsecuencias comunes a X e Y , pero no son las más largas.
 $\langle B, D, A, B \rangle$ y $\langle B, C, B, A \rangle$ son de largo 4 y son las subsecuencias comunes de mayor tamaño entre X e Y .

Longest Common Subsequent Problem

Sean $X = \langle X_1, X_2, \dots, X_m \rangle$ e $Y = \langle Y_1, Y_2, \dots, Y_n \rangle \in \Sigma^*$

- El problema de encontrar la subsecuencia común más larga, consiste precisamente en determinar una subsecuencia común a X e Y y que sea de largo máximo.
- Note que existen 2^m posibles subsecuencias para X y 2^n posibles subsecuencias para Y .
Esto se puede ver, por ejemplo, si para X disponemos de un bitstream $B[1..m]$, en el cual marcamos $B[i] = 1$ si X_i esta presente en la secuencia o con un 0 si no lo esta, y sabemos que en m bits hay 2^m posibles números.
- Este problema se puede resolver por Búsqueda Exhaustiva (fuerza bruta), que desde luego tarda tiempo exponencial. Esto es buscar cada una de las 2^m subsecuencias de X en Y , o buscar las 2^n subsecuencias de Y en X .

LCS problem - Búsqueda Exhaustiva

Input: Ambas subsecuencias $X[1..m]$ e $Y[1..n]$

Output: El largo máximo de una LCS entre X e Y .

```
LCS-Exhaustiva( $X, m, Y, n$ ) {  
    if ( $m == 0 \vee n == 0$ ) then  
        return 0  
    if ( $X[m] == Y[n]$ ) then  
        return 1 + LCS-Exhaustiva( $X, m - 1, Y, n - 1$ )  
    return max(LCS-Exhaustiva( $X, m - 1, Y, n$ ),  
              LCS-Exhaustiva( $X, m, Y, n - 1$ ))  
}
```

- Peor caso: $T(m, n) = 1 + T(m - 1, n) + T(m, n - 1)$

- A modo de comparación, recuerde Hanoi:

$$H(k) = 1 + 2H(k - 1) = O(2^k)$$

LCS problem - Búsqueda Exhaustiva

Input: Ambas subsecuencias $X[1..m]$ e $Y[1..n]$

Output: El largo máximo de una LCS entre X e Y .

```
LCS-Exhaustiva( $X, m, Y, n$ ) {  
    if ( $m == 0 \vee n == 0$ ) then  
        return 0  
    if ( $X[m] == Y[n]$ ) then  
        return  $1 + \text{LCS-Exhaustiva}(X, m - 1, Y, n - 1)$   
    return max( $\text{LCS-Exhaustiva}(X, m - 1, Y, n)$ ,  
               $\text{LCS-Exhaustiva}(X, m, Y, n - 1)$ )  
}
```

- Peor caso: $T(m, n) = 1 + T(m - 1, n) + T(m, n - 1)$

- A modo de comparación, recuerde Hanoi:

$$H(k) = 1 + 2H(k - 1) = O(2^k)$$

- Suponga entonces que $T(m, n) \approx H(m + n) = O(2^{m+n})$

- Desde luego, los casos exponenciales en la práctica son muy malos y los queremos evitar $\Rightarrow$ usaremos PD.

LCS problem - Programación Dinámica

Caracterización del Problema: utilizaremos el siguiente principio invariante.

Teorema. Sean $X = \langle X_1, \dots, X_m \rangle$ e $Y = \langle Y_1, \dots, Y_n \rangle \in \Sigma^*$ y $Z = \langle Z_1, \dots, Z_k \rangle = \text{LCS}(X, Y)$, entonces:

- i- Si $X_m = Y_n \Rightarrow Z_k = X_m = Y_n$ y $Z_{1..k-1} = \text{LCS}(X_{1..m-1}, Y_{1..n-1})$.
- ii- Si $X_m \neq Y_n$ y $Z_k \neq X_m \Rightarrow Z_{1..k} = \text{LCS}(X_{1..m-1}, Y_{1..n})$.
- iii- Si $X_m \neq Y_n$ y $Z_k \neq Y_n \Rightarrow Z_{1..k} = \text{LCS}(X_{1..m}, Y_{1..n-1})$.

Demostración.

i- $X_m = Y_n$, suponga que $Z_k \neq X_m \Rightarrow Z' = Z_{1..k} \cdot X_m$ es secuencia común entre X e Y , con $|Z'| > |Z| \rightarrow \leftarrow$ ya que $Z = \text{LCS}(X, Y)$.

- ii- Tenemos que $X_m \neq Y_n$ y $Z_k \neq X_m$, y suponga $Z_{1..k} \neq \text{LCS}(X_{1..m-1}, Y)$
 $\Rightarrow$ como $Z = \text{LCS}(X, Y)$ y $Z_k \neq X_m \Rightarrow Z = \text{CS}(X_{1..m-1}, Y)$
 $\Rightarrow \exists a \in \Sigma$ tq., al insertar o concat. a en Z , $Z' = Z \cdot a = \text{LCS}(X_{1..m-1}, Y)$
pero $Z = \text{LCS}(X, Y)$ y ahora $|Z'| > |Z| = k$ ya que $Z' = \text{LCS}(X_{1..m-1}, Y)$
 $\rightarrow \leftarrow$ Contradicción !!

No puede existir $Z' = \text{CS}(X, Y)$ tal que $|Z'| > k = |\text{LCS}(X, Y)|$

- iii- Se demuestra reciprocamente a (ii-).

LCS problem - Programación Dinámica

Haciendo uso del teorema anterior, nos planteamos la ecuación de recurrencia dinámica en función de una matriz $C[1..m, 1..n]$, en donde almacenaremos en $C[i, j] = \text{LCS}(X_{1..i}, Y_{1..j})$. Entonces:

$$C[i, j] = \begin{cases} 0 & \text{Si } i = 0 \vee j = 0 \\ 1 + C[i - 1, j - 1] & \text{Si } X_i = Y_j \\ \max(C[i - 1, j], C[i, j - 1]) & \text{Si } X_i \neq Y_j \end{cases}$$

Note que a diferencia del Rod Cutting, el valor del óptimo es condicional.

Reconocemos aquí las 4 etapas típicas en PD para este tipo de problemas de optimización:

1. **Catacterización:** definir la estructura de una solución óptima, lo que hicimos a través del teorema
2. **Solución recursiva:** dada por medio de la matriz C
3. **Calcular óptimos locales:** $C[i, j] = 0$ cuando $i = 0$ o $j = 0$; y $C[i, j] = C[i - 1, j - 1] + 1$ si $X_i = Y_j$
4. **Construir el óptimo global:** Calcular el largo de un LCS $\Rightarrow$ Algoritmo $\text{LCS-LENGTH}(X, Y)$ que no es otra cosa que aplicar la ecuación recursiva definida en 2.

LCS - Algoritmo

Input: Ambas subsecuencias $X[1..m]$ e $Y[1..n]$

Output: El largo máximo de un LCS entre X e Y .

```
LCS-LENGTH( $X, m, Y, n$ ) {  
    Sean  $B[1..m, 1..n]$  y  $C[0..m, 0..n]$  matrices.  
    for ( $i = 0$  to  $m$ ) do  
         $C[i, 0] = 0$   
    for ( $j = 1$  to  $n$ ) do  
         $C[0, j] = 0$   
    for ( $i = 1$  to  $m$ ) do {  
        for ( $j = 1$  to  $n$ ) do  
            if ( $X[i] == Y[j]$ ) then {  
                 $C[i, j] = 1 + C[i - 1, j - 1]$   
                 $B[i, j] = \nwarrow$   
            } else if ( $C[i - 1, j] \geq C[i, j - 1]$ ) then {  
                 $C[i, j] = C[i - 1, j]$   
                 $B[i, j] = \uparrow$   
            } else {  
                 $C[i, j] = C[i, j - 1]$   
                 $B[i, j] = \leftarrow$   
            }  
        }  
    }  
    return( $C, B$ )  
}
```


LCS - Algoritmo

Input: Ambas subsecuencias $X[1..m]$ e $Y[1..n]$

Output: El largo máximo de un LCS entre X e Y .

```
LCS-LENGTH( $X, m, Y, n$ ) {  
    Sean  $B[1..m, 1..n]$  y  $C[0..m, 0..n]$  matrices.  
    for ( $i = 0$  to  $m$ ) do  
         $C[i, 0] = 0$   
    for ( $j = 1$  to  $n$ ) do  
         $C[0, j] = 0$   
    for ( $i = 1$  to  $m$ ) do {  
        for ( $j = 1$  to  $n$ ) do  
            if ( $X[i] == Y[j]$ ) then {  
                 $C[i, j] = 1 + C[i - 1, j - 1]$   
                 $B[i, j] = \nwarrow$   
            } else if ( $C[i - 1, j] \geq C[i, j - 1]$ ) then {  
                 $C[i, j] = C[i - 1, j]$   
                 $B[i, j] = \uparrow$   
            } else {  
                 $C[i, j] = C[i, j - 1]$   
                 $B[i, j] = \leftarrow$   
            }  
        }  
    }  
    return( $C, B$ )  
}
```

$\Rightarrow T(m, n) = O(m \cdot n)$

LCS - Algoritmo - Ejemplo

		j	0	1	2	3	4	5	6
		y_j		B	D	C	A	B	A
i	x_i								
0			0	0	0	0	0	0	0
1	A		0	↑	↑	↑	↖1	←1	↖1
2	B		0	↖1	←1	←1	↑1	↖2	←2
3	C		0	↑1	↑1	↖2	←2	↑2	↑2
4	B		0	↖1	↑1	↑2	↑2	↖3	←3
5	D		0	↑1	↖2	↑2	↑2	↑3	↑3
6	A		0	↑1	↑2	↑2	↖3	↑3	↖4
7	B		0	↖1	↑2	↑2	↑3	↖4	↑4

LCS - Algoritmo

Para imprimir la LCS nos basta cruzar la matriz B desde $B[m, n]$ hasta $B[1, 1]$, siguiendo la dirección de las flechas e imprimir el símbolo $X[i]$ cada vez que encontremos $B[i, j] = \nwarrow$. Esto lo realizamos invocando a PRINT-LCS con $i = m$ y $j = n$:

```
PRINT-LCS( $B, X, i, j$ ){  
    if ( $i == 0 \vee j == 0$ ) then  
        return  
    if ( $B[i, j] == \nwarrow$ ) then  
        PRINT-LCS( $B, X, i - 1, j - 1$ )  
        imprimir( $X[i]$ )  
    else if ( $B[i, j] == \uparrow$ ) then  
        PRINT-LCS( $B, X, i - 1, j$ )  
    else  
        PRINT-LCS( $B, X, i, j - 1$ )  
}
```

LCS - Mejoras

Ejercicio

Si solamente se le pide el largo de la $LCS(X, Y)$, ¿Cómo podría modificar el algoritmo $LCS-LENGTH()$ para ahorrar espacio, no utilizando las matrices C y B sino (a lo más) un par de arreglos de largo n o m ?

Diseñe este algoritmo y escriba el pseudocódigo.

Inducción - Maximum Subarray Sum Problem (MSSP)

El problema se define de la siguiente manera:

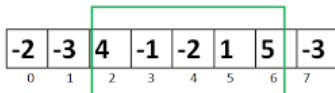
Dado un arreglo numerico $A = [1..n]$, encontrar a, b ;

$1 \leq a \leq b \leq n$ tal que:

$$\sum_{j=a}^b A[j], \text{ sea máxima.}$$

Ejemplo:

Largest Subarray Sum Problem



$$4 + (-1) + (-2) + 1 + 5 = 7$$

Maximum Contiguous Array Sum is 7

MSSP - Fuerza bruta

```
MAX-SS-BruteForce( $A, n$ ){  
     $t = 0$   
    for ( $i = 1$  to  $n$ ) do {  
         $S = 0$   
        for ( $j = i$  to  $n$ ) do {  
             $S = S + A[j]$   
            if ( $S > t$ ) then {  
                 $t = S$   
                 $a = i$   
                 $b = j$   
            }  
        }  
    }  
    if ( $t == 0$ ) then {  
         $a = b = \text{findIndexOfMax}(A, n)$   
         $t = A[a]$   
    }  
    return( $t, a, b$ )  
}
```

¿Complejidad?

MSSP - Fuerza bruta

```
MAX-SS-BruteForce( $A, n$ ){  
     $t = 0$   
    for ( $i = 1$  to  $n$ ) do {  
         $S = 0$   
        for ( $j = i$  to  $n$ ) do {  
             $S = S + A[j]$   
            if ( $S > t$ ) then {  
                 $t = S$   
                 $a = i$   
                 $b = j$   
            }  
        }  
    }  
    if ( $t == 0$ ) then {  
         $a = b = \text{findIndexOfMax}(A, n)$   
         $t = A[a]$   
    }  
    return( $t, a, b$ )  
}
```

$\rightarrow O(n)$

$\rightarrow O(n)$

$\rightarrow O(n)$

¿Complejidad? $\Rightarrow$ Claramente $T(n) = O(n^2)$

Resolver MSSP Mediante Divide y Vencerás

Como entrada tenemos al arreglo $A[\ell \dots r]$. Luego:

- Dividiremos el arreglo en dos partes: la partición de la izquierda $A[\ell \dots m]$ y la partición de la derecha $A[m + 1 \dots r]$, para un índice m en el centro: $m = (\ell + r)/2$
- Resolvemos los subproblemas de ambas particiones (para $A[\ell \dots m]$ y $A[m + 1 \dots r]$), y nos quedamos con la suma máxima entre:
 - A la izquierda $\rightarrow \text{MSSP}(A[\ell \dots m])$
 - A la derecha $\rightarrow \text{MSSP}(A[m + 1 \dots r])$
 - Que incluye sobrelapamiento $\rightarrow \text{MSSP}(A[\ell' \dots r'])$, con $\ell' \leq m$ y $r' > m$
 $\Rightarrow$ habrá que buscar cuales son estos límites ℓ' y r'

MSSP - Divide y Vencerás - Pseudocódigo

```
MAX-SS-DV( $A, \ell, r$ ) {  
    if ( $\ell == r$ ) return  $A[\ell]$   
     $m = (\ell + r) / 2$   
    return max(MAX-SS-DV( $A, \ell, m$ ), Cross-MAX-SS-DV( $A, \ell, m, r$ ),  
              MAX-SS-DV( $A, m + 1, r$ ))  
}
```

```
Crossing-MAX-SS-DV( $A, \ell, m, r$ ) {  
     $S_\ell = t_\ell = A[m]$   
    for ( $i = m - 1$  to  $\ell$ ) do { //  $i$  disminuye de 1 en 1  
         $S_\ell = S_\ell + A[i]$   
        if ( $S_\ell > t_\ell$ )  $t_\ell = S_\ell$   
    }  
     $S_r = t_r = A[m + 1]$   
    for ( $i = m + 2$  to  $r$ ) do {  
         $S_r = S_r + A[i]$   
        if ( $S_r > t_r$ )  $t_r = S_r$   
    }  
    return  $t_\ell + t_r$   
}
```

⇒ ¿Cuánto es tiempo total, $T(n)$?

MSSP - Divide y Vencerás - Pseudocódigo

```
MAX-SS-DV( $A, \ell, r$ ) {  
    if ( $\ell == r$ ) return  $A[\ell]$   
     $m = (\ell + r) / 2$   
    return max(MAX-SS-DV( $A, \ell, m$ ), Cross-MAX-SS-DV( $A, \ell, m, r$ ),  
              MAX-SS-DV( $A, m + 1, r$ ))  
}
```

```
Crossing-MAX-SS-DV( $A, \ell, m, r$ ) {  
     $S_\ell = t_\ell = A[m]$   
    for ( $i = m - 1$  to  $\ell$ ) do {                                //  $i$  disminuye de 1 en 1  
         $S_\ell = S_\ell + A[i]$   
        if ( $S_\ell > t_\ell$ )  $t_\ell = S_\ell$   
    }  
     $S_r = t_r = A[m + 1]$   
    for ( $i = m + 2$  to  $r$ ) do {  
         $S_r = S_r + A[i]$   
        if ( $S_r > t_r$ )  $t_r = S_r$   
    }  
    return  $t_\ell + t_r$   
}
```

⇒ ¿Cuánto es tiempo total, $T(n)$?

$$\rightarrow T(n) = 2T(n/2) + \Theta(n)$$

MSSP - Divide y Vencerás - Pseudocódigo

```
MAX-SS-DV( $A, \ell, r$ ) {  
    if ( $\ell == r$ ) return  $A[\ell]$   
     $m = (\ell + r)/2$   
    return max(MAX-SS-DV( $A, \ell, m$ ), Cross-MAX-SS-DV( $A, \ell, m, r$ ),  
              MAX-SS-DV( $A, m + 1, r$ ))  
}
```

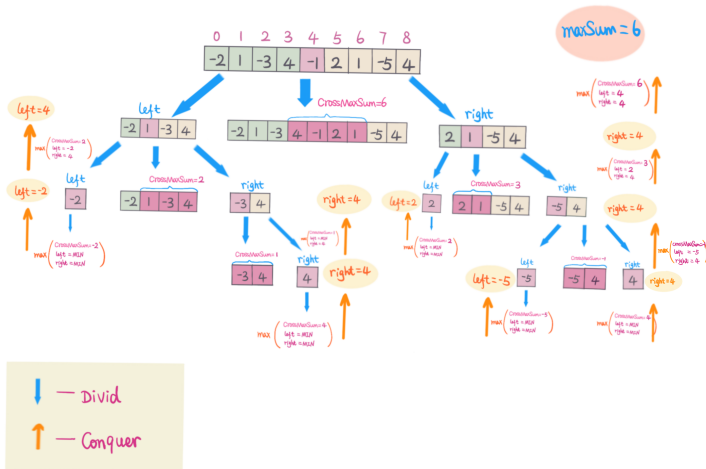
```
Crossing-MAX-SS-DV( $A, \ell, m, r$ ) {  
     $S_\ell = t_\ell = A[m]$   
    for ( $i = m - 1$  to  $\ell$ ) do { //  $i$  disminuye de 1 en 1  
         $S_\ell = S_\ell + A[i]$   
        if ( $S_\ell > t_\ell$ )  $t_\ell = S_\ell$   
    }  
     $S_r = t_r = A[m + 1]$   
    for ( $i = m + 2$  to  $r$ ) do {  
         $S_r = S_r + A[i]$   
        if ( $S_r > t_r$ )  $t_r = S_r$   
    }  
    return  $t_\ell + t_r$   
}
```

⇒ ¿Cuánto es tiempo total, $T(n)$?

$$\rightarrow T(n) = 2T(n/2) + \Theta(n) = O(n \log n)$$

También su árbol de recurrencia (binario) es perfectamente equilibrado, donde **Crossing-MAX-SS-DV**(A, ℓ, m, r) cuesta $O(r - \ell)$, por tanto será $O(n)$ en cada nivel al igual que en quicksort en su mejor caso.

Segunda forma de Resolver MSSP con Divide y Vencerás



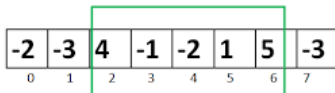
Ejercicio: escriba el pseudocódigo. Note que el elemento central es una partición por si solo.

Técnica de Inducción

- Técnica de diseño que intenta replicar el proceso de inducción matemática.
- La metodología se acomoda solo a un pequeño conjunto de problemas.
- Son problemas en los que es fácil acumular, o llevar o retener, una solución parcial al problema para cualquier prefijo de la entrada $1...k$ y utilizar la solución de este prefijo para determinar la solución en el paso $k + 1$.
- Típicamente, el proceso, para una entrada $n \in \mathbb{N}$, consiste en iterar en $k = 1, 2, \dots, n$ y resolver el problema para cada k , utilizando la solución disponible en $k - 1$.

Resolver MSSP Mediante Inducción

Largest Subarray Sum Problem



$$4 + (-1) + (-2) + 1 + 5 = 7$$

Maximum Contiguous Array Sum is 7

Tal como en inducción matemática:

Supondremos resuelto el problema para $A[1..k - 1]$ y lo usaremos para resolver $A[1..k]$.

Algoritmo MAX-SS-Induction:

Input: El arreglo $A[1..n]$

Output: La suma máxima t y sus límites a, b en $A[a..b]$

Algoritmo MAX-SS-Induction

```
MAX-SS-Induction( $A, n$ ){  
     $t = S = 0$   
     $i = 1$   
    for ( $j = 1$  to  $n$ ) do {  
         $S = S + A[j]$   
        if ( $S \leq 0$ ) then {  
             $S = 0$   
             $i = j + 1$   
        }  
        if ( $S > t$ ) then {  
             $t = S$   
             $a = i$   
             $b = j$   
        }  
    }  
    if ( $t = 0$ ) then {  
         $a = b = \text{findIndexOfMax}(A, n)$   
         $t = A[a]$   
    }  
    return( $t, a, b$ )  
}
```

¿Cuál es la complejidad $T(n)$?

Algoritmo MAX-SS-Induction

```
MAX-SS-Induction( $A, n$ ){  
     $t = S = 0$   
     $i = 1$   
    for ( $j = 1$  to  $n$ ) do {  
         $S = S + A[j]$   $\rightarrow O(n)$   
        if ( $S \leq 0$ ) then {  
             $S = 0$   
             $i = j + 1$   
        }  
        if ( $S > t$ ) then {  
             $t = S$   
             $a = i$   
             $b = j$   
        }  
    }  
    if ( $t = 0$ ) then {  
         $a = b = \text{findIndexOfMax}(A, n)$   $\rightarrow O(n)$   
         $t = A[a]$   
    }  
    return( $t, a, b$ )  
}
```

¿Cuál es la complejidad $T(n)$? $\Rightarrow$ Claramente $T(n) = O(n)$